Journal of Statistical Software

MMMM YYYY, Volume VV, Issue II. *doi: 10.18637/jss.v000.i00*

surveyframe: A Proactive Survey Research Workflow for R

Mohammed Ali Sharafuddin

Abstract

Survey-based research in the social sciences, management, and health disciplines typically follows a reactive pattern: an instrument is built, data collected, and statistical techniques applied retrospectively. This approach leaves the analysis plan unspecified until after data collection, creating conditions for researcher degrees of freedom, post-hoc model fitting, and irreproducible findings.

surveyframe introduces a *proactive* workflow architecture built around the `sframe` class, a typed, validated R object that encodes the complete instrument specification, including item definitions, response choice sets, scale membership, branching logic, attention checks, and a pre-declared analysis plan, before a single response is collected. Analysis is the execution of a pre-declared methodological contract, not a search through analytical space.

The package provides a constructor family for building `sframe` instruments, SHA-256-integrity-checked serialisation to the `.sframe` format, browser-based and **Shiny**-based survey rendering, CSV-based and Google Sheets response collection, a multi-stage quality-checking pipeline, scale scoring with reverse coding, Cronbach's α and McDonald's ω reliability statistics, EFA suitability diagnostics (KMO, Bartlett), lavaan CFA syntax generation, a pre-declared analysis plan executor producing APA-formatted results and interpretation prompts, and reproducible HTML report generation. All core functions are available without optional dependencies; statistical extensions activate gracefully when **psych** or **lavaan** are installed.

This article describes the design philosophy, the package architecture, and demonstrates the complete workflow using a bundled tourism services survey instrument and response dataset.

Keywords: survey research, psychometrics, instrument design, reproducible research, R, pre-registration.

1. Introduction

Survey-based research is one of the most widely used methods across the social sciences, management, education, and health disciplines. A researcher designs an instrument, collects responses, and then applies statistical procedures to the resulting data. In the dominant tooling ecosystem (SPSS (IBM Corp. 2023), SmartPLS (Ringle, Wende, and Becker 2022), **lavaan** (Rosseel 2012)), the statistical analysis is a distinct, post-hoc step performed after data are in hand. The instrument, the analysis method, and the interpretation criteria are often determined or refined only after examining the data.

This reactive pattern creates conditions that are now well-documented in the reproducibility literature (Simmons, Nelson, and Simonsohn 2011; Open Science Collaboration 2015). Researchers may, intentionally or not, adjust their scale definitions, select among competing fit indices, or revise their theoretical model after observing the data. Even when no scientific misconduct is involved, the statistical consequences of such flexibility are severe: inflated Type I error rates, overfit models, and findings that do not replicate (Gelman and Loken 2014).

Pre-registration disciplines the analysis plan by requiring researchers to declare hypotheses, variables, and analysis procedures before data collection (Nosek, Ebersole, DeHaven, and Mellor 2018). A growing number of journals and funders now mandate or incentivise pre-registration. Yet most statistical software provides no mechanisms to enforce or even express such a plan. The analysis plan exists in a separate document, is manually followed, and is never checked against the actual analysis executed.

surveyframe addresses this gap with a *proactive workflow* architecture. The central abstraction is the `sframe` class: a typed R object that encodes the instrument definition *and* the intended analysis plan in a single, validated, and integrity-checked object. The sframe is constructed before data collection. The analysis is the execution of the plan embedded in the instrument, not a new plan written after inspecting results. When a researcher serialises an `sframe` to disk with `write_sframe()`, an SHA-256 hash over the entire specification is stored alongside it. If the instrument is later modified after data collection, the stored hash provides an auditable record of the change.

This design is not a policy mechanism; researchers can always construct new analysis objects to run additional tests. What it provides is *visibility*: the pre-declared analysis is clearly separated from any post-hoc exploration, making the distinction legible in both the code and the final report.

surveyframe covers the complete survey research lifecycle: instrument construction, validation, deployment as a self-contained static HTML file or **Shiny** application, response collection and import, data quality checking, scale scoring, reliability analysis, EFA suitability diagnostics, lavaan CFA syntax generation, pre-declared analysis plan execution, codebook generation, and HTML report rendering. The package has three hard dependencies (**jsonlite**, **rlang**, **openssl**) and activates statistical extensions gracefully when **psych** or **lavaan** are present.

The remainder of this article is organised as follows. Section 2 describes the design philosophy and the `sframe` class. Section 3 demonstrates instrument construction. Section 4 covers survey deployment and response collection. Section 5 presents the analysis workflow. Section 6 describes reporting. Section 7 compares **surveyframe** with related software. Section 8 concludes.

2. Design philosophy and the `sframe` class

2.1. The proactive workflow

The key architectural decision in **surveyframe** is that an `sframe` object is always constructed from scratch, without data. It describes an instrument and an intended analysis. Data are then collected, imported, and passed into analysis functions alongside the `sframe` as a second argument. The analysis functions read the declared structure (scale membership, reverse coding, item types, and analysis plan) from the instrument, not from the data.

This is in contrast to the typical R workflow in which a researcher begins with a data frame and then constructs variable sets, scale definitions, and model specifications interactively. In **surveyframe**, those decisions must be made and committed to the `sframe` before any data are in scope.

The practical consequence is that an sframe file produced before data collection serves as a machine-readable proxy for a pre-registration. When submitted to a pre-registration registry (e.g., OSF) alongside a timestamp, or when used to derive the JSS replication file hash, the `.sframe` file constitutes evidence that the analysis plan was fixed before the results were observed.

2.2. The `sframe` class

An `sframe` object is a named list with eight typed slots:

`meta` Character fields: `title`, `version`, `description`, `authors`, `languages`, and `validated`.

`items` A list of `sf_item` objects, one per survey item.

`choices` A list of `sf_choices` objects, one per response scale.

`scales` A list of `sf_scale` objects defining composite scoring rules and measurement structure.

`branching` A list of `sf_branch` objects encoding conditional display logic.

`checks` A list of `sf_check` objects defining attention-check criteria.

`analysis_plan` A list of pre-declared analysis blocks, each specifying a test type, variable roles, and decision rules.

`models` A list of `sf_model` objects encoding structural relationships between constructs for CFA/SEM syntax generation.

The `sframe` is constructed via the `sf_instrument()` function, which accepts a `components` list of objects produced by the `sf_` constructor family. Components are sorted into their respective slots automatically; passing a non-constructor object raises a validation error.

2.3. Integrity and serialisation

`write_sframe()` serialises an `sframe` to the `.sframe` format (a compressed JSON container) and computes an SHA-256 hash over the serialised content. The hash is stored in the file

header alongside the package version. `read_sframe()` recomputes the hash on load and warns if the stored and computed hashes differ, indicating that the file was modified after serialisation. This hash is the primary audit mechanism for detecting post-collection instrument editing.

3. Instrument construction

3.1. Constructor family

Five constructors produce the typed components assembled into an `sframe`:

`sf_choices()` Defines a named response scale (choice set) mapping numeric values to display labels. Choice sets are referenced by `id` in item definitions and reused across items sharing the same scale.

`sf_item()` Defines a single survey item. Supports thirteen item types: closed-response types (`"likert"`, `"single_choice"`, `"multiple_choice"`, `"matrix"`, `"slider"`, `"ranking"`, `"rating"`), open-response types (`"numeric"`, `"text"`, `"textarea"`, `"date"`), and layout elements (`"section_break"`, `"text_block"`).

`sf_scale()` Groups items into a composite scale and specifies scoring rules: `method` (`"mean"` or `"sum"`), `min_valid` (minimum non-missing items for a valid score), `reverse_items`, and optional item `weights`.

`sf_branch()` Defines a conditional display rule: an item is shown only when a specified condition on a prior item's response is met.

`sf_check()` Defines an attention-check criterion used by `quality_report()` to flag inattentive respondents.

3.2. Building an instrument

The following code constructs a minimal tourism services survey with two constructs and five items, based on the instrument used throughout the worked example in this article.

```
library("surveyframe")

## Define a shared 5-point agreement response scale
agree5 <- sf_choices(
  id     = "agree5",
  values = 1:5,
  labels = c("Strongly disagree", "Disagree", "Neutral",
             "Agree", "Strongly agree")
)

## Define items: declare scale membership before any data exist
```

```
dm_1 <- sf_item("dm_1",
  "Social media content influences my travel decisions.",
  type = "likert", choice_set = "agree5",
  scale_id = "digital_marketing", required = TRUE)

dm_2 <- sf_item("dm_2",
  "Online reviews shape my destination choices.",
  type = "likert", choice_set = "agree5",
  scale_id = "digital_marketing", required = TRUE)

dm_3 <- sf_item("dm_3",
  "Digital promotions encourage me to visit new places.",
  type = "likert", choice_set = "agree5",
  scale_id = "digital_marketing", required = TRUE)

sq_1 <- sf_item("sq_1",
  "The hospitality staff were attentive and helpful.",
  type = "likert", choice_set = "agree5",
  scale_id = "service_quality", required = TRUE)

sq_2 <- sf_item("sq_2",
  "Facilities were clean and well maintained.",
  type = "likert", choice_set = "agree5",
  scale_id = "service_quality", required = TRUE)

## Define scales - scoring rule committed before data collection
dm_scale <- sf_scale(
  id     = "digital_marketing",
  label  = "Digital marketing effectiveness",
  items  = c("dm_1", "dm_2", "dm_3"),
  method = "mean"
)

sq_scale <- sf_scale(
  id     = "service_quality",
  label  = "Service quality",
  items  = c("sq_1", "sq_2"),
  method = "mean"
)

## Assemble the instrument
instr <- sf_instrument(
  title       = "Tourism Experience Survey",
  version     = "1.0.0",
  description = "Digital marketing and service quality perceptions.",
  authors     = "J. Researcher",
```

```
  components  = list(agree5, dm_1, dm_2, dm_3, sq_1, sq_2,
                     dm_scale, sq_scale)
)
print(instr)

## <sframe>
##   Title:      Tourism Experience Survey
##   Version:    1.0.0
##   Items:      5
##   Scales:     2
##   Status:     not validated
```

3.3. Validation

`validate_sframe()` checks the internal consistency of an `sframe` object and reports all problems. The function performs more than twenty checks, including duplicate item IDs, items referencing missing choice sets, scale items not present in the instrument, branching rules referencing unknown items, and analysis plan roles referencing missing variables. In `strict = TRUE` mode (the default), any problem raises a typed error of class `sframe_validation_error`. In `strict = FALSE` mode, all problems are returned as a character vector without stopping.

```
result <- validate_sframe(instr, strict = FALSE)
result$valid

## [1] TRUE

result$problems

## character(0)
```

3.4. Serialisation

`write_sframe()` serialises a validated `sframe` to the `.sframe` format and records an SHA-256 hash of the instrument specification:

```
R> write_sframe(instr, "tourism_survey.sframe")
R> instr2 <- read_sframe("tourism_survey.sframe")
R> identical(instr$meta$title, instr2$meta$title)
[1] TRUE
```

The hash verification on `read_sframe()` detects out-of-band modifications made after the file was created. Any change to the serialised JSON, including edits to item text, scale membership, or the analysis plan, produces a hash mismatch, which is reported as a warning.

3.5. Declaring the analysis plan

An `sframe` optionally carries a pre-declared analysis plan as a list of analysis blocks in the `analysis_plan` slot. Blocks specify the statistical test type, variable roles, options (e.g., α level), and decision rules. The bundled demonstration instrument includes a full plan; the worked example in Section 5 runs it.

Analysis plans can be authored either programmatically or through the visual SurveyBuilder interface described in Section 4. When constructed through the interface, the plan is stored in the `.sframe` file and retrieved by `run_analysis_plan()` without any additional specification.

4. Survey deployment and response collection

4.1. Static HTML export

`export_static_survey()` generates a self-contained single-file HTML survey that runs entirely in the browser without a server or internet connection. All thirteen item types, branching logic, required-field validation, multi-page navigation, and progress indication are implemented in client-side JavaScript. When a respondent submits the form, the browser downloads a one-row CSV file. If an `endpoint_url` is supplied (a Google Apps Script web app or a serverless function), the response is also POSTed as JSON. The CSV download proceeds regardless of POST success, so no response is lost on network failure.

```
R> export_static_survey(instr,
+    output_path = "tourism_survey.html",
+    open = FALSE)
```

The exported file can be hosted on any static file server, GitHub Pages, Netlify, or sent as an email attachment. No server or R installation is required on the respondent's machine.

4.2. Visual SurveyBuilder

`launch_builder()` opens a browser-based visual instrument editor. Researchers who prefer a graphical interface can build the complete `sframe` (items, choice sets, scales, branching rules, and analysis plan) without writing R code. The builder exports the completed `.sframe` file; the file is imported into R with `read_sframe()` for analysis.

4.3. Shiny survey module

`render_survey()` launches an interactive **Shiny** survey. For embedding within a larger application, `survey_module_ui()` provides the UI component and `survey_module_server()` provides the server logic, returning a reactive that holds the response list once the form is submitted.

4.4. Response import

`read_responses()` reads a CSV file of responses, aligns item columns against the instrument definition, and adds metadata columns for respondent ID, submission timestamp, and any

supplementary fields. The function validates that all declared item columns are present and warns if the column types are inconsistent with item definitions.

```
responses <- read_responses(
  x             = "responses.csv",
  instrument    = instr,
  respondent_id = "respondent_id",
  submitted_at  = "submitted_at"
)
```

For the worked example that follows, the bundled demonstration dataset is loaded directly:

```
demo  <- sframe_demo_data()
instr <- demo$instrument
resp  <- demo$responses

## Five scales, 15 items, 120 respondents
cat("Scales:", length(instr$scales), "\n")

## Scales: 5

cat("Items:", length(instr$items), "\n")

## Items: 15

cat("Responses:", nrow(resp), "\n")

## Responses: 120
```

5. The analysis workflow

surveyframe structures the analysis workflow as a sequence of discrete, composable steps. Each function accepts a data frame and an `sframe` instrument as its primary arguments and returns a typed object with a `print()` method. Steps can be executed independently or as a complete pre-declared analysis plan.

5.1. Data quality report

`quality_report()` performs five classes of quality checks on the response data, using the instrument definition to determine what constitutes a valid response:

1. **Attention checks.** Flags respondents who fail declared `sf_check()` criteria embedded in the instrument.
2. **Timing.** When a start-time column is present, flags respondents who completed the survey faster than a declared minimum duration.

3. **Straight-lining.** Flags respondents who gave the same response to every Likert item.
4. **Missing data.** Reports per-item and per-respondent missingness rates.
5. **Duplicates.** Detects duplicate respondent IDs.

```
qr <- quality_report(resp, instr)
cat("Respondents:", qr$summary$n_respondents, "\n")

## Respondents: 120

cat("Items:      ", qr$summary$n_items, "\n")

## Items:       15

cat("Flagged:    ", qr$summary$n_flagged, "\n")

## Flagged:     109
```

5.2. Scale scoring

`score_scales()` computes composite scale scores for every respondent according to the scoring rules declared in the `sframe`. Reverse coding is applied automatically using either the per-item `reverse` flag in `sf_item()` or the `reverse_items` vector in `sf_scale()`; both sources are respected. The `min_valid` argument controls missingness tolerance: respondents with fewer valid items than the declared threshold receive `NA` for that scale.

```
scored <- score_scales(resp, instr)
## Scale score columns appended to the response data frame
scale_cols <- c("digital_marketing", "service_quality",
                "sustainability", "satisfaction",
                "behavioural_intention")
summary(scored[, scale_cols])

##  digital_marketing service_quality sustainability  satisfaction
##  Min.   :1.00      Min.   :1.33    Min.   :1.50   Min.   :1.00
##  1st Qu.:2.67      1st Qu.:2.33    1st Qu.:2.50   1st Qu.:2.50
##  Median :3.00      Median :3.00    Median :3.00   Median :3.25
##  Mean   :3.15      Mean   :3.06    Mean   :3.19   Mean   :3.29
##  3rd Qu.:3.67      3rd Qu.:3.67    3rd Qu.:4.00   3rd Qu.:4.00
##  Max.   :5.00      Max.   :5.00    Max.   :5.00   Max.   :5.00
##  behavioural_intention
##  Min.   :1.00
##  1st Qu.:2.50
##  Median :3.00
```

```
##  Mean   :3.10
##  3rd Qu.:3.62
##  Max.   :5.00
```

5.3. Reliability analysis

`reliability_report()` computes Cronbach's α (Cronbach 1951) and McDonald's ω (McDonald 1999) for each scale in the instrument, using **psych** (Revelle 2024) when available. The function reads scale definitions directly from the `sframe`, requiring no manual specification of item vectors.

```
if (requireNamespace("psych", quietly = TRUE)) {
  rr <- reliability_report(resp, instr, omega = FALSE)
  print(rr)
}

## Reliability Report
##
## Scale: digital_marketing (Digital marketing effectiveness)
##   Items: 3   N: 120
##   Alpha:   0.837
##
## Scale: service_quality (Service quality)
##   Items: 3   N: 120
##   Alpha:   0.844
##
## Scale: sustainability (Sustainability perception)
##   Items: 2   N: 120
##   Alpha:   0.772
##
## Scale: satisfaction (Tourist satisfaction)
##   Items: 2   N: 120
##   Alpha:   0.816
##
## Scale: behavioural_intention (Behavioural intention)
##   Items: 2   N: 120
##   Alpha:   0.844
```

All five scales exceed the conventional $\alpha \geq 0.70$ threshold (Nunnally 1978), supporting internal consistency.

5.4. EFA suitability

Before exploratory factor analysis, **surveyframe** evaluates dataset suitability through the Kaiser-Meyer-Olkin (KMO) measure, Bartlett's test of sphericity, and parallel analysis via `efa_report()`:

```
R> if (requireNamespace("psych", quietly = TRUE)) {
+    er <- efa_report(resp, instr)
+    print(er)
+  }
```

The function constructs the item matrix from columns identified in the instrument, passes it to `psych::KMO()`, `psych::cortest.bartlett()`, and `psych::fa.parallel()`, ensuring only the declared measurement items are included.

5.5. CFA syntax generation

`cfa_syntax()` generates lavaan (Rosseel 2012) CFA model syntax directly from the scale definitions in the `sframe`. Each scale becomes a reflective latent construct with its declared items as indicators. The output is ready to pass to `lavaan::cfa()` when **lavaan** is installed:

```
syn <- cfa_syntax(instr)
cat(syn)

## # lavaan CFA syntax generated by surveyframe
## # Model: Tourism Services Experience Demo CFA
## # Recommended fitting option: std.lv = TRUE
## # Fit with lavaan only when lavaan is installed.
##
## # Digital marketing effectiveness (reflective)
## digital_marketing =~ dm_1 + dm_2 + dm_3
##
## # Service quality (reflective)
## service_quality =~ sq_1 + sq_2 + sq_3
##
## # Sustainability perception (reflective)
## sustainability =~ sus_1 + sus_2
##
## # Tourist satisfaction (reflective)
## satisfaction =~ sat_1 + sat_2
##
## # Behavioural intention (reflective)
## behavioural_intention =~ bi_1 + bi_2
```

The syntax encodes the measurement model declared before data collection. Because the construct–indicator relationships were specified in `sf_scale()` at instrument construction time, the CFA tests a pre-declared structure rather than one selected after inspecting inter-item correlations.

5.6. Running the analysis plan

`run_analysis_plan()` executes all analysis blocks declared in the `sframe`'s `analysis_plan` slot and returns an object of class `sframe_analysis_results`. Each result carries the test

statistic, p value, effect size, an APA-formatted summary string, and a `prompt` field containing a plain-language interpretation template that the researcher completes:

```
results <- run_analysis_plan(resp, instr, scored = TRUE)

## Correlation: digital marketing and satisfaction
r1 <- results[[1]]
cat("Test:  ", r1$test, "\n")

## Test:   correlation_pearson

cat("APA:   ", r1$apa, "\n")

## APA:    r(118) = 0.54, p < .001

cat("Effect:", r1$effect_label, "\n")

## Effect: large

cat("Q:", strwrap(r1$research_question, width = 58,
                  exdent = 3), sep = "\n    ")

## Q:
##     Is perceived digital marketing effectiveness associated
##        with tourist satisfaction?

cat("\n")
```

The `prompt` field explicitly signals that the interpretation is the researcher's responsibility: **surveyframe** provides the statistic and its label; the researcher supplies the substantive interpretation. This design discourages the conflation of statistical and practical significance that is endemic in automated reporting tools.

The second analysis block in the bundled plan is a multiple regression:

```
r2 <- results[[2]]
cat("Test:", r2$test, "\n")

## Test: regression_linear

cat("APA: ", r2$apa, "\n")

## APA:  R² = 0.383, F(3, 116) = 23.95, p < .001
```

The third block tests whether first-time and repeat visitors differ in behavioural intention using a Mann-Whitney U test, appropriate because `visit_type` is a binary categorical predictor and `behavioural_intention` has a restricted range:

```
r3 <- results[[3]]
cat("Test:", r3$test, "\n")

## Test: mann_whitney

cat("APA: ", r3$apa, "\n")

## APA:  U = 1576, z = -0.98, p = 0.327, r = 0.09

cat("Effect:", r3$effect_label, "\n")

## Effect: negligible
```

Supported analysis types include Pearson and Spearman correlations, independent and paired t tests, Mann-Whitney U, Kruskal-Wallis, one-way and two-way ANOVA with Tukey HSD, ANCOVA, repeated-measures ANOVA, simple linear regression, logistic regression (binary, ordinal, and multinomial), partial correlation, and mediation and moderation templates. Each test type carries a curated citation list sourced from the `.sframe_citations` registry, providing the researcher with the primary methodological references for their methods section.

6. Reporting

`render_report()` produces a self-contained HTML report from the analysis results, instrument metadata, and codebook. The report includes an executive summary, per-scale reliability statistics, analysis plan results with APA-formatted tables, the full codebook, and a citation block listing all packages and methodological references used in the analysis. All output is contained in a single HTML file that can be shared without external dependencies.

```
R> render_report(
+    instrument  = instr,
+    data        = resp,
+    output_file = "tourism_report.html"
+  )
```

`codebook_report()` generates a standalone codebook listing all items, their types, choice sets, scale membership, and reverse-coding status. The codebook can be exported as HTML or Markdown for inclusion in supplementary materials or ethics board submissions.

```
cb <- codebook_report(instr)
nrow(cb$items_table)

## [1] 15

head(cb$items_table[, c("id", "type", "scale_id", "reverse")], 6)
```

Table 1: Comparison of **surveyframe** with related software. ✓ = available; – = not available; P = planned.

Feature	**surveyframe**	SPSS	SmartPLS	**lavaan**	jamovi	Qualtrics
Instrument construction	✓	✓	–	–	–	✓
Pre-declared analysis plan	✓	–	–	–	–	–
SHA-256 instrument hash	✓	–	–	–	–	–
Static HTML survey export	✓	–	–	–	–	–
Shiny survey module	✓	–	–	–	–	–
Scale scoring (mean/sum)	✓	✓	✓	–	✓	–
Reliability (α, ω)	✓	✓	✓	–	✓	–
EFA suitability (KMO, Bartlett)	✓	✓	–	–	✓	–
CFA syntax generation	✓	–	–	✓	✓	–
CB-SEM execution	P	✓	–	✓	✓	–
PLS-SEM execution	P	–	✓	–	–	–
APA results with citations	✓	–	–	–	–	–
HTML report generation	✓	–	–	–	✓	✓
CRAN open source	✓	–	–	✓	✓	–
Self-hostable	✓	–	–	✓	✓	–
No external calls	✓	✓	✓	✓	✓	–

```
##           id          type          scale_id reverse
## 1 visit_type single_choice                     FALSE
## 2       dm_1        likert digital_marketing   FALSE
## 3       dm_2        likert digital_marketing   FALSE
## 4       dm_3        likert digital_marketing   FALSE
## 5       sq_1        likert   service_quality   FALSE
## 6       sq_2        likert   service_quality   FALSE
```

The citation block in `render_report()` automatically includes citations for every test type run in the analysis plan, sourced from the internal `.sframe_citations` registry.

7. Comparison with existing software

Table 1 summarises **surveyframe** alongside the principal existing tools for survey-based quantitative research.

surveyframe's primary distinction is the *proactive* architecture: instrument construction, analysis plan declaration, and data collection are strictly ordered. All other tools in Table 1 operate on data that already exist; none require or provide mechanisms for encoding the analysis plan in the instrument before data collection. This is the feature that most directly addresses the reproducibility concerns outlined in Section 1.

SPSS. IBM SPSS Statistics (IBM Corp. 2023) provides a comprehensive GUI-based analysis environment widely used in social science and management research. SPSS includes survey collection tools (through associated products), reliability analysis, factor analysis, and regression. It does not provide R-native scripting, open-source licensing, or an instrument-first workflow. Annual licensing costs are substantial, creating access barriers for individual researchers and lower-income institution contexts.

SmartPLS. SmartPLS (Ringle *et al.* 2022) specialises in partial least squares structural equation modelling (PLS-SEM). It is widely adopted in management research, particularly for formative constructs and small sample sizes. SmartPLS does not support instrument construction, pre-declared analysis plans, or open-source access. It has no R API; analyses are performed through its own proprietary interface. CB-SEM and lavaan CFA are outside its scope. PLS-SEM execution is on the **surveyframe** development roadmap, with syntax generation for **seminr** already available in v0.3.0.

lavaan. The **lavaan** package (Rosseel 2012) is the standard R tool for structural equation modelling. It accepts model syntax defined by the researcher and operates on existing data frames. **surveyframe** complements rather than competes with **lavaan**: the `cfa_syntax()` function generates lavaan syntax directly from the `sframe`'s scale definitions, providing the measurement model specification that the researcher then passes to **lavaan**. The critical difference is that **surveyframe** constrains the CFA to the constructs declared before data collection; **lavaan** itself imposes no such constraint.

jamovi. The jamovi project (The jamovi project 2022) provides a free, open-source GUI for statistical analysis with an active R API through **jmv**. jamovi covers descriptives, reliability, EFA, and CFA (via **lavaan**). It does not provide instrument construction, a static HTML survey exporter, or a pre-declared analysis plan facility. It is a close analogue for the analysis and reporting portion of **surveyframe**'s workflow.

8. Summary

surveyframe introduces a proactive architecture for survey-based research in R. The `sframe` class encodes the complete research instrument (items, response scales, composite scale definitions, branching logic, and analysis plan) in a single validated and integrity-checked object constructed before data collection. This design makes the distinction between a pre-declared analysis and post-hoc exploration explicit in the code and the final report, directly supporting the methodological standards demanded by reproducible-research initiatives and pre-registration mandates.

The package covers the complete survey research lifecycle: instrument construction and validation, SHA-256-integrity-checked serialisation, browser-based and Shiny-based survey rendering, static HTML export for offline or server-free deployment, CSV and Google Sheets response collection, a multi-stage data quality pipeline, scale scoring with automatic reverse coding, Cronbach's α and McDonald's ω reliability statistics, EFA suitability diagnostics, lavaan CFA syntax generation, a pre-declared analysis plan executor with APA-formatted results and interpretation prompts, codebook generation, and HTML report rendering.

surveyframe v0.3.0 is available on CRAN. Planned extensions include PLS-SEM and CB-SEM execution via **seminr** and **lavaan**, item response theory workflows, multi-criteria decision making, and Quarto-native report generation. The package repository and issue tracker are available at https://github.com/MohammedAliSharafuddin/surveyframe.

References

Cronbach LJ (1951). "Coefficient Alpha and the Internal Structure of Tests." *Psychometrika*, **16**(3), 297–334. doi:10.1007/BF02310555.

Gelman A, Loken E (2014). "The Statistical Crisis in Science." *American Scientist*, **102**(6), 460–465. doi:10.1511/2014.111.460.

IBM Corp (2023). *IBM SPSS Statistics for Windows, Version 29.0*. IBM Corp., Armonk, NY. URL https://www.ibm.com/products/spss-statistics.

McDonald RP (1999). *Test Theory: A Unified Treatment*. Lawrence Erlbaum, Mahwah, NJ.

Nosek BA, Ebersole CR, DeHaven AC, Mellor DT (2018). "The Preregistration Revolution." *Proceedings of the National Academy of Sciences*, **115**(11), 2600–2606. doi:10.1073/pnas.1708274114.

Nunnally JC (1978). *Psychometric Theory*. 2nd edition. McGraw-Hill, New York.

Open Science Collaboration (2015). "Estimating the Reproducibility of Psychological Science." *Science*, **349**(6251), aac4716. doi:10.1126/science.aac4716.

Revelle W (2024). *psych: Procedures for Psychological, Psychometric, and Personality Research*. Northwestern University, Evanston, Illinois. R package version 2.4.3, URL https://CRAN.R-project.org/package=psych.

Ringle CM, Wende S, Becker JM (2022). *SmartPLS 4*. SmartPLS, Boenningstedt, Germany. URL https://www.smartpls.com.

Rosseel Y (2012). "lavaan: An R Package for Structural Equation Modeling." *Journal of Statistical Software*, **48**(2), 1–36. doi:10.18637/jss.v048.i02.

Simmons JP, Nelson LD, Simonsohn U (2011). "False-Positive Psychology: Undisclosed Flexibility in Data Collection and Analysis Allows Presenting Anything as Significant." *Psychological Science*, **22**(11), 1359–1366. doi:10.1177/0956797611417632.

The jamovi project (2022). *jamovi (Version 2.3) [Computer Software]*. URL https://www.jamovi.org.

A. Session information

```
toLatex(sessionInfo())
```

- R version 4.6.0 (2026-04-24), `x86_64-pc-linux-gnu`
- Locale: `LC_CTYPE=en_GB.UTF-8`, `LC_NUMERIC=C`, `LC_TIME=en_IN.UTF-8`, `LC_COLLATE=en_GB.UTF-8`, `LC_MONETARY=en_IN.UTF-8`, `LC_MESSAGES=en_GB.UTF-8`, `LC_PAPER=en_IN.UTF-8`, `LC_NAME=C`, `LC_ADDRESS=C`, `LC_TELEPHONE=C`, `LC_MEASUREMENT=en_IN.UTF-8`, `LC_IDENTIFICATION=C`
- Time zone: `Asia/Kolkata`

- TZcode source: `system (glibc)`
- Running under: `Ubuntu 24.04.4 LTS`
- Matrix products: default
- BLAS: `/usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0`
- LAPACK: `/usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0`
- Base packages: base, datasets, graphics, grDevices, methods, stats, utils
- Other packages: knitr 1.51, surveyframe 0.3.0
- Loaded via a namespace (and not attached): askpass 1.2.1, cli 3.6.6, compiler 4.6.0, evaluate 1.0.5, grid 4.6.0, highr 0.12, jsonlite 2.0.0, lattice 0.22-9, mnormt 2.1.2, nlme 3.1-169, openssl 2.4.0, otel 0.2.0, parallel 4.6.0, psych 2.6.3, rlang 1.2.0, tools 4.6.0, xfun 0.57

B. Replication

All results in this manuscript are reproducible using the `replicate.R` script included with the submission. The script loads **surveyframe**, calls `sframe_demo_data()` to obtain the bundled instrument and response dataset, and executes every code example shown above in sequence. No additional data files or network access are required.

Affiliation:

Mohammed Ali Sharafuddin
E-mail: mohammedali.page@gmail.com
ORCID: 0000-0001-5247-2964

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMM YYYY, Volume VV, Issue II *Submitted:* 2026-05-22
doi:10.18637/jss.v000.i00 *Accepted:* yyyy-mm-dd